

## Лекция 2. Формальные грамматики, иерархия Хомского, язык Милан

**Курс:** Информационные системы поддержки принятия решений

**Источники:** фото конспекта (3 листа): иерархия Хомского, грамматика учебного языка, пример программы и таблица её выполнения.

[дополнение] — добавлено мной для понимания; [неясно] — не восстанавливается однозначно (см. конец файла).

### Ключевые определения и теоремы

| Понятие                                                | Суть                                                                                                                                                                                                                                                             |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Формальная грамматика</b> [дополнение: определение] | Набор правил вида «левая часть $\rightarrow$ правая часть», по которым из начального символа порождаются слова языка. Заглавные буквы ( $A, B, \dots$ ) — нетерминалы (вспомогательные символы), строчные ( $a, b, \dots$ ) — терминалы (буквы итогового слова). |
| $\Lambda$                                              | Пустое слово.                                                                                                                                                                                                                                                    |
| <b>Тип 0</b>                                           | Правила $v \rightarrow w, v \neq \Lambda$ . Грамматики без ограничений.                                                                                                                                                                                          |
| <b>Тип 1</b>                                           | Правила $uAv \rightarrow u w v, w \neq \Lambda$ .<br>Контекстно-зависимые.                                                                                                                                                                                       |
| <b>Тип 2</b>                                           | Правила $A \rightarrow w$ . Контекстно-свободные (КС).<br>Распознаются МП-автоматами.                                                                                                                                                                            |
| <b>Тип 3</b>                                           | Правила $A \rightarrow aB, A \rightarrow a, A \rightarrow \Lambda$ . Автоматные (регулярные). Распознаются конечными автоматами.                                                                                                                                 |
| <b>Иерархия Хомского</b>                               | $K_3 \subsetneq K_2 \subsetneq K_1 \subsetneq K_0$ , где $K_i$ — класс языков, порождаемых грамматиками типа $i$ .                                                                                                                                               |
| <b>Язык Милан</b>                                      | Учебный язык программирования, заданный КС-грамматикой (п. 3).                                                                                                                                                                                                   |

### 1. Что такое грамматика (коротко) [дополнение]

Грамматика — способ **конечным** набором правил описать **бесконечный** язык. Начинаем с начального символа и шаг за шагом заменяем левую часть какого-нибудь правила на его правую часть. Когда нетерминалов не осталось, получилось слово языка.

Пример: правила  $A \rightarrow aA, A \rightarrow b$  порождают  $b, ab, aab, \dots$  — то есть язык  $\{a^n b \mid n \geq 0\}$ .

Иерархия Хомского сортирует грамматики по тому, **насколько свободными могут быть правила**. Чем строже ограничение, тем проще устройство, которое язык распознаёт.

## 2. Иерархия Хомского

| Тип | Вид правил                                                 | Название грамматик                   | Распознающее устройство                        |
|-----|------------------------------------------------------------|--------------------------------------|------------------------------------------------|
| 0   | $v \rightarrow w, v \neq \Lambda$                          | Без ограничений                      | машины Тьюринга, нормальные алгорифмы Маркова* |
| 1   | $uAv \rightarrow uvw, w \neq \Lambda$                      | Контекстно-зависимые                 | *см. «Спорные места»                           |
| 2   | $A \rightarrow w$                                          | Контекстно-свободные (КС-грамматики) | МП-автоматы (с магазинной памятью)             |
| 3   | $A \rightarrow aB, A \rightarrow a, A \rightarrow \Lambda$ | Автоматные (регулярные)              | конечный автомат                               |

\* В конспекте «Машины Тьюринга / Нормальные алгорифмы Маркова» стоят в строке типа 1, а клетка типа 0 пуста — подробнее в «Спорных местах».

### Как понимать правила:

- **Тип 1:** нетерминал  $A$  можно заменить на  $w$  только «в окружении»  $u \dots v$  — замена зависит от контекста.
- **Тип 2:**  $A$  заменяется на  $w$  всегда, независимо от соседей — «свободно от контекста».
- **Тип 3:** справа не больше одного нетерминала, и стоит он в конце: слово растёт строго слева направо. Это ровно поведение конечного автомата: прочитал букву  $a$  — перешёл в состояние  $B$ .

### Включение классов языков:

$$K_3 \subsetneq K_2 \subsetneq K_1 \subsetneq K_0.$$

Каждый следующий класс шире, и включения строгие.



Иерархия Хомского: классы языков вложены друг в друга (примеры языков — [дополнение])

**[дополнение] Примеры для строгости:**  $\{a^n b^n \mid n \geq 1\}$  — КС-язык ( $S \rightarrow aSb \mid ab$ ), но не регулярный: конечный автомат не может «посчитать» сколько угодно букв  $a$ .  $\{a^n b^n c^n\}$  — контекстно-зависимый, но не КС.

### 3. Язык Милан

[неясно] В конспекте написано « $L_3$  — язык Ми...ан»; я читаю это как **Милан** — известный учебный язык с ровно такими конструкциями (`begin...end`, `while...do...od`, `if...then...else...fi`). Его грамматика — контекстно-свободная (тип 2).

#### 3.1. Грамматика

```

П → begin L end
L → S | S ; L
S → id := E
    | while B do L od
    | if B then L else L fi
    | write ( E )
B → E Q E
Q → = | < | > | >= | <= | <>
E → E + T | E - T | T
T → T * P | T / P | P
P → id | const | ( E ) | read

```

**Что означают нетерминалы** [дополнение: названия]:

| Символ    | Смысл                                                                             |
|-----------|-----------------------------------------------------------------------------------|
| П         | программа                                                                         |
| L         | список операторов через ;                                                         |
| S         | оператор: присваивание, цикл, ветвление, вывод                                    |
| B         | условие (сравнение двух выражений)                                                |
| Q         | знак сравнения (<> — «не равно»)                                                  |
| E, T, P   | выражение, слагаемое, множитель                                                   |
| id, const | идентификатор (имя переменной) и числовая константа                               |
| read      | чтение числа с ввода; это <b>выражение</b> , поэтому пишут <code>m := read</code> |

**[дополнение] Почему E, T, P устроены «лесенкой».** Так грамматика сама задаёт приоритет операций: \* и / «глубже» (в T), чем + и - (в E), поэтому  $a + b * c$  разбирается как  $a + (b * c)$ . Левая рекурсия  $E \rightarrow E + T$  задаёт левую ассоциативность:  $a - b - c = (a - b) - c$ .

#### 3.2. Пример программы: алгоритм Евклида (вычитанием)

```

begin
  m := read;
  n := read;

```

```

while m <> n do
  if m > n then
    m := m - n
  else
    n := n - m
  fi;
od;
write(m)
end

```

**Что делает программа.** Находит наибольший общий делитель (НОД) чисел  $m$  и  $n$ . Идея [дополнение: обоснование]:  $\gcd(m, n) = \gcd(m - n, n)$  при  $m > n$  — общий делитель  $m$  и  $n$  делит и разность. Из большего числа вычитаем меньшее, пока числа не сравняются; общее значение и есть НОД.

#### 4. Семантика: выполнение программы по шагам

В конспекте этот блок озаглавлен «Денотационная семантика языков программирования» (о названии см. «Спорные места»). Пример — трассировка программы на входе  $m = 6, n = 3$ :

| $m$ | $n$ | Комментарий                                                                                     |
|-----|-----|-------------------------------------------------------------------------------------------------|
| 6   | 0   | $m := \text{read}$ (прочитали 6; $n$ ещё не прочитано)                                          |
| 6   | 3   | $n := \text{read}$                                                                              |
| 6   | 3   | проверка цикла: $6 \neq 3 \rightarrow$ входим в цикл                                            |
| 6   | 3   | проверка if: $6 > 3 \rightarrow$ ветка then                                                     |
| 3   | 3   | $m := m - n$                                                                                    |
| 3   | 3   | проверка цикла: $3 = 3 \rightarrow$ выход из цикла<br>$\text{write}(3) \rightarrow$ результат 3 |

Ответ  $\gcd(6, 3) = 3$  — верно.

Ещё один пример для тренировки [дополнение]:  $m = 12, n = 18$ .

| $m$ | $n$ | Шаг                                          |
|-----|-----|----------------------------------------------|
| 12  | 18  | $12 \neq 18, 12 < 18 \rightarrow n := n - m$ |
| 12  | 6   | $12 \neq 6, 12 > 6 \rightarrow m := m - n$   |
| 6   | 6   | $6 = 6 \rightarrow$ выход, $\text{write}(6)$ |

$\gcd(12, 18) = 6$ .

#### Спорные и неоднозначные места

1. **Название языка.** В тетради «Ми...ан» (буквы плохо различимы). **Гипотеза: Милан** — учебный язык из курсов по теории трансляции; синтаксис полностью совпадает.
2. « $L_3$  — язык Милан». Индекс 3 не объяснён. Грамматика Милана — КС (тип 2), а не тип 3, так что это вряд ли номер типа. **Гипотеза:** просто обозначение языка или номер примера. [неясно]

3. **Распознающие устройства для типов 0 и 1.** В тетради «машины Тьюринга / нормальные алгоритмы Маркова» записаны в строке типа 1, а у типа 0 клетка пустая. **[дополнение]** **Стандартно:** тип 0  $\leftrightarrow$  машины Тьюринга (и эквивалентные им алгоритмы Маркова), тип 1  $\leftrightarrow$  линейно ограниченные автоматы. **Гипотеза:** запись относилась к типу 0 и просто «съехала» по строке, либо преподаватель не стал различать.
4.  **$A \rightarrow \Lambda$  в типе 3 и условие  $w \neq \Lambda$  в типе 1.** Формально правило с пустой правой частью запрещено в типе 1, поэтому включение  $K_3 \subseteq K_1$  верно «с точностью до пустого слова». **[дополнение]** Это стандартная оговорка, в конспекте её нет.
5. **fi; и od; в программе.** По грамматике  $L \rightarrow S \mid S; L$  после последнего оператора списка ; не ставится, а в программе есть fi; перед od и od; перед write. od; корректно (далее идёт write), а fi; перед od формально даёт пустой оператор, которого в грамматике нет. **Гипотеза:** вольность записи на доске; на смысл не влияет. Кроме того, в правиле if в тетради написано fi; — точка с запятой там, скорее всего, лишняя.
6. **«Денотационная семантика» и трассировка.** Пошаговое выполнение с таблицей значений ближе к **операционной** семантике. Денотационная семантика сопоставляет программе математический объект — функцию «вход  $\rightarrow$  выход» (здесь gcd). **Гипотеза:** заголовок открывал тему семантики целиком, а трассировка — иллюстрация; в следующей лекции идёт «процедурная семантика». Рукопись в заголовке читается неуверенно.
7. **Правило  $B \rightarrow E \ Q \ E$ .** В тетради последний символ похож на  $\bar{E}/E=$ ; принял  $E \ Q \ E$  — по смыслу это единственный вариант.

## Вопросы для самопроверки

1. Чем терминалы отличаются от нетерминалов?
2. Назовите четыре типа грамматик по Хомскому, вид их правил и распознающие устройства.
3. Почему в грамматике типа 3 нетерминал справа может стоять только в конце?
4. Приведите язык, который КС, но не регулярный. Почему конечный автомат его не распознаёт?
5. Как грамматика Милана обеспечивает, что \* выполняется раньше +?
6. Выведите из грамматики Милана оператор  $x := x + 1$ , начиная с S.
7. Протрассируйте программу Евклида на входе  $m = 10, n = 4$ . Какой будет результат?
8. Что произойдёт, если на вход подать  $m = 5, n = 0$ ? (Подсказка: остановится ли цикл?)